



EC-220 Computer System Architecture

Dr. Ahsan Shahzad

Department of Computer and Software Engineering (DC&SE)

College of Electrical and Mechanical Engineering (CEME)

National University of Science and Technology (NUST)

Week # 15_OLE

Pipeline Hazards

■ Pipeline Hazards

- Situations in pipelining when the next instruction cannot execute in the following clock cycle. Those events are called *hazards*.
- There are **3 different types**

1) Structural Hazards

- When a planned instruction cannot execute in the proper clock cycle because the hardware does not support the combination of instructions that are set to execute. E.g. Washer-Dryer Combination

2) Data Hazards

- Also called a pipeline data hazard. When a planned instruction cannot execute in the proper clock cycle because data that is needed to execute the instruction is not yet available.

3) Control Hazards

- Also called branch hazard. ... because the instruction that was fetched is not the one that is needed; that is, the flow of instruction addresses is not what the pipeline expected.

Control Hazards

■ Control Hazard

- arising from the need to make a decision based on the results of one instruction while others are executing.
- E.g. conditional branches

■ Solutions

- Stall
 - Stop the pipeline immediately after branch until you know the address where the next instruction will be fetch.
- Predict
 - Static Prediction
 - Dynamic Prediction
- Delayed Branches / Delayed Decision
 - By scheduling and executing the next instruction of the branch.

Control Hazards:

- Assume, Branch condition true

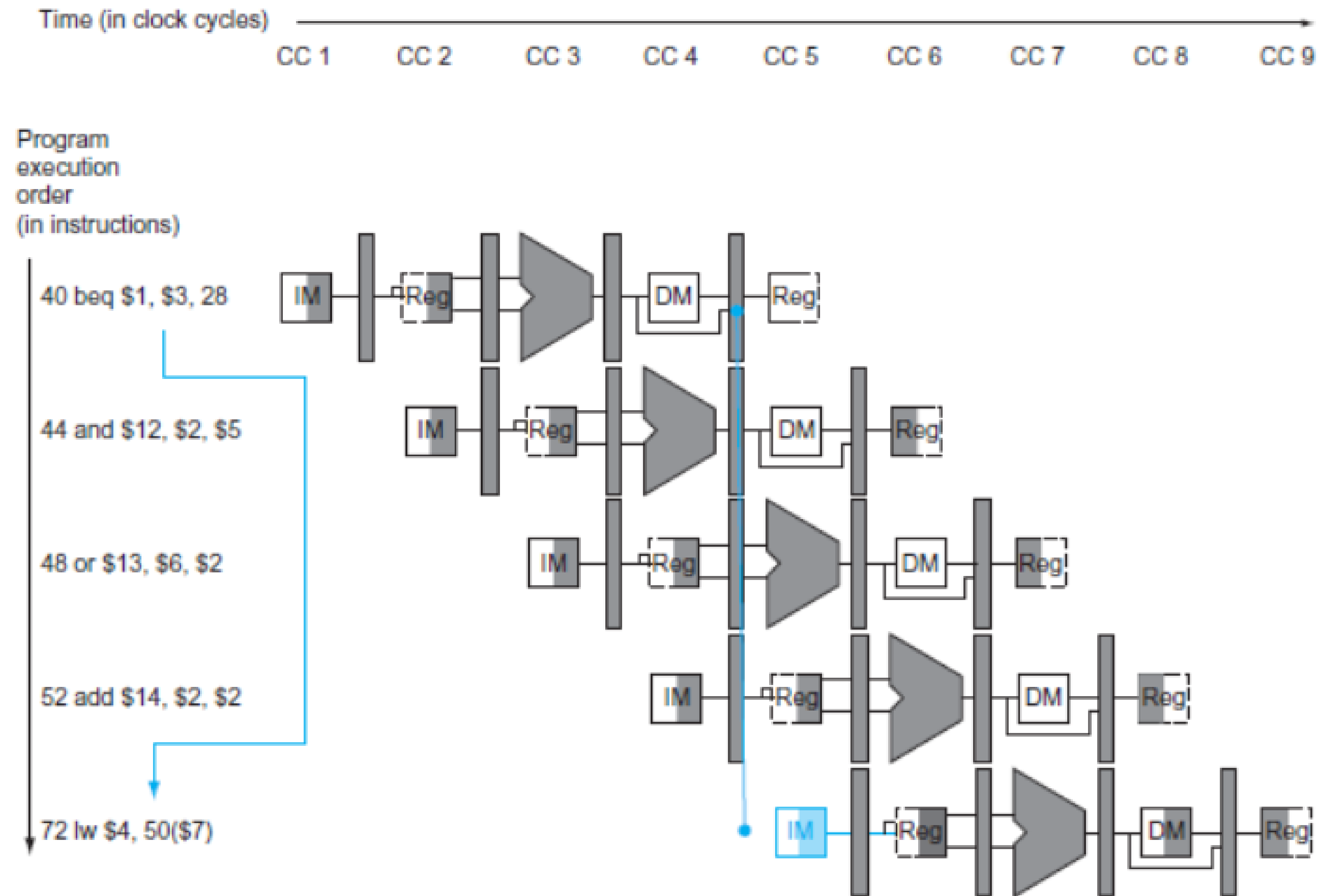


FIGURE 4.61 The Impact of the pipeline on the branch instruction. The numbers to the left of the instruction (40, 44, ...) are the addresses of the instructions. Since the branch instruction decides whether to branch in the MEM stage—clock cycle 4 for the beq instruction above—the three sequential instructions that follow the branch will be fetched and begin execution. Without intervention, those three following instructions will begin execution before beq branches to lw at location 72. (Figure 4.31 assumed extra hardware to reduce the control hazard to one clock cycle; this figure uses the nonoptimized datapath.)

Control Hazards: 1) Stall the Pipeline

- Assume, branch target address calculation, Registers testing and PC update can be done at 2nd Stage.

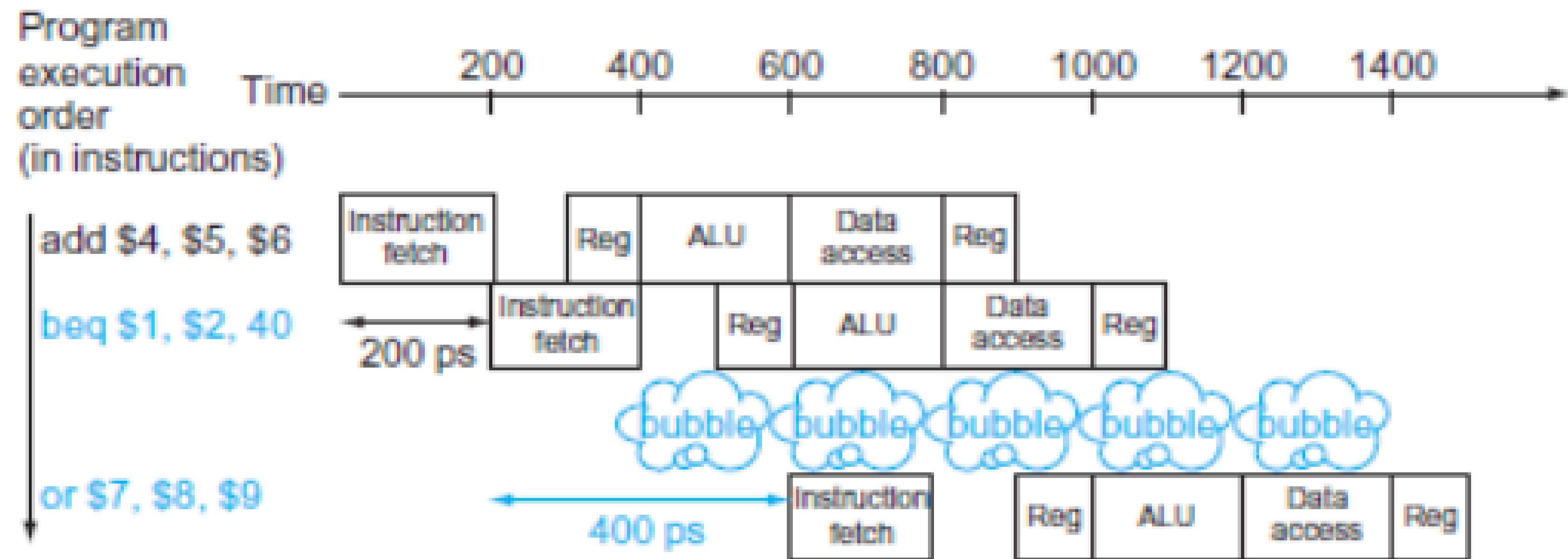


FIGURE 4.31 Pipeline showing **stalling on every conditional branch** as solution to control hazards. This example assumes the conditional branch is taken, and the instruction at the destination of the branch is the OR instruction. There is a one-stage pipeline stall, or bubble, after the branch. **In reality, the process of creating a stall is slightly more complicated, as we will see in Section 4.8.** The effect on performance, however, is the same as would occur if a bubble were inserted.

Control Hazards: 2) Predict

- **Static Prediction**

- **Assume Branch Not Taken**
- Continue execution down the sequential instruction stream.
- If Branch is taken, then the instructions that are fetched and decoded after branch must be discarded.
- E.g. if 50% branches are taken and cost to discard the instrs. will be little. We are saving half the cost of control hazards.
- *Difference with Load-use data hazard Stall:*
 - Making control signals zero at ID stage to discard the instruction or insert bubble.
 - Now we need to remove three instructions in the IF, ID and EX stage
 - **Flush:** To discard instructions in a pipeline usually due to an unexpected event.

Control Hazards: 2) Predict – Branch Not Taken

- **Reducing the Delay of Branches**

- Move the branch execution earlier in the pipeline, then fewer instrs. need to be flushed.
- i.e. Computing the branch target address and Evaluating the branch decision Early.
- **Branch Target Address Calculation**
 - Easy part, move the hardware from EX stage to ID stage.
- **Branch Decision**
 - Harder part, Simple ALU needs to be added. E.g. for Equality Testing -> XOR respective bits of inputs and then ORing the results.
 - For this ALU / Equality Testing Hardware, additional forwarding and hazard detection hardware is required at ID stage.

Control Hazards: 2) Predict – Branch Not Taken (Ctd.)

Reducing the Delay of Branches

▪ Branch Decision

- Two complicating factors related to forwarding results to Equality Test Logic at ID stage.
 - During ID, decode the instruction, decide whether a bypass to the equality unit is needed, and complete the equality comparison if the instr. is branch. Update PC with branch target address. **Require New fwding unit** at ID stage.
 - **Possible Data Hazard-** Stall the pipeline
 - E.g. ALU (R-type) instr. Immediately preceding by a branch instr. Stall
 - Load-use scenario in branch: stall for 2 clock cycles.

- Despite these difficulties moving branch to the ID stage is an improvement, penalty of branch reduces from 3 to 1 instruction if branch is taken else zero.
- IF.Flush: clear the instruction field of the IF/ID pipeline register, generate nop instr.

Control Hazards: 2) Predict – Branch Not Taken (Ctd.)

Example: Pipelined Branch

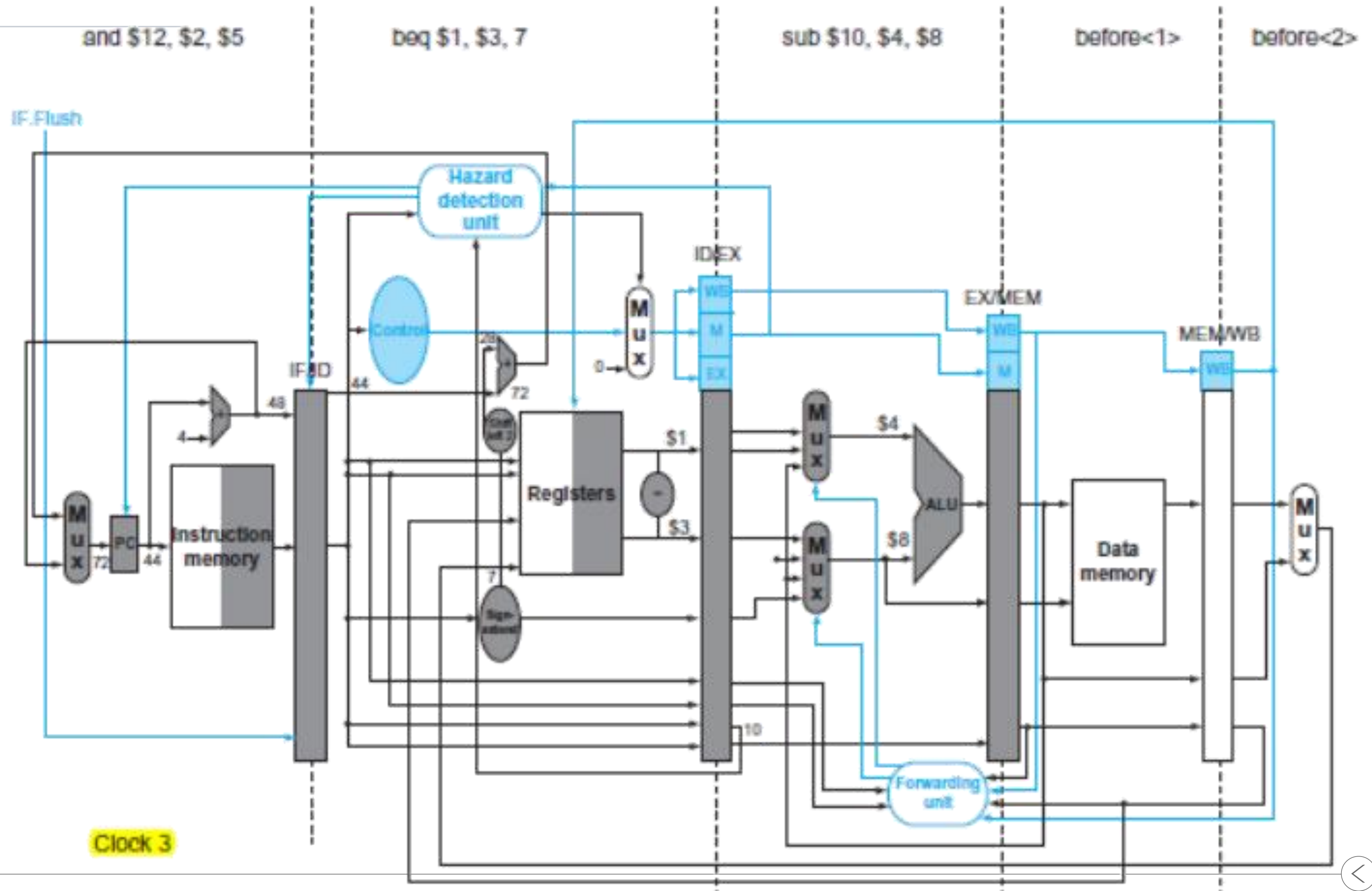
Show what happens when the branch is taken in this instruction sequence, assuming the pipeline is optimized for branches that are not taken and that we moved the branch execution to the ID stage:

```
36 sub $10, $4, $8
40 beq $1, $3, 7 # PC-relative branch to 40+4+7*4=72
44 and $12, $2, $5
48 or  $13, $2, $6
52 add $14, $4, $2
56 slt $15, $6, $7
. . .
72 lw  $4, 50($7)
```

Figure 4.62 shows what happens when a branch is taken. Unlike Figure 4.61, there is only one pipeline bubble on a taken branch.

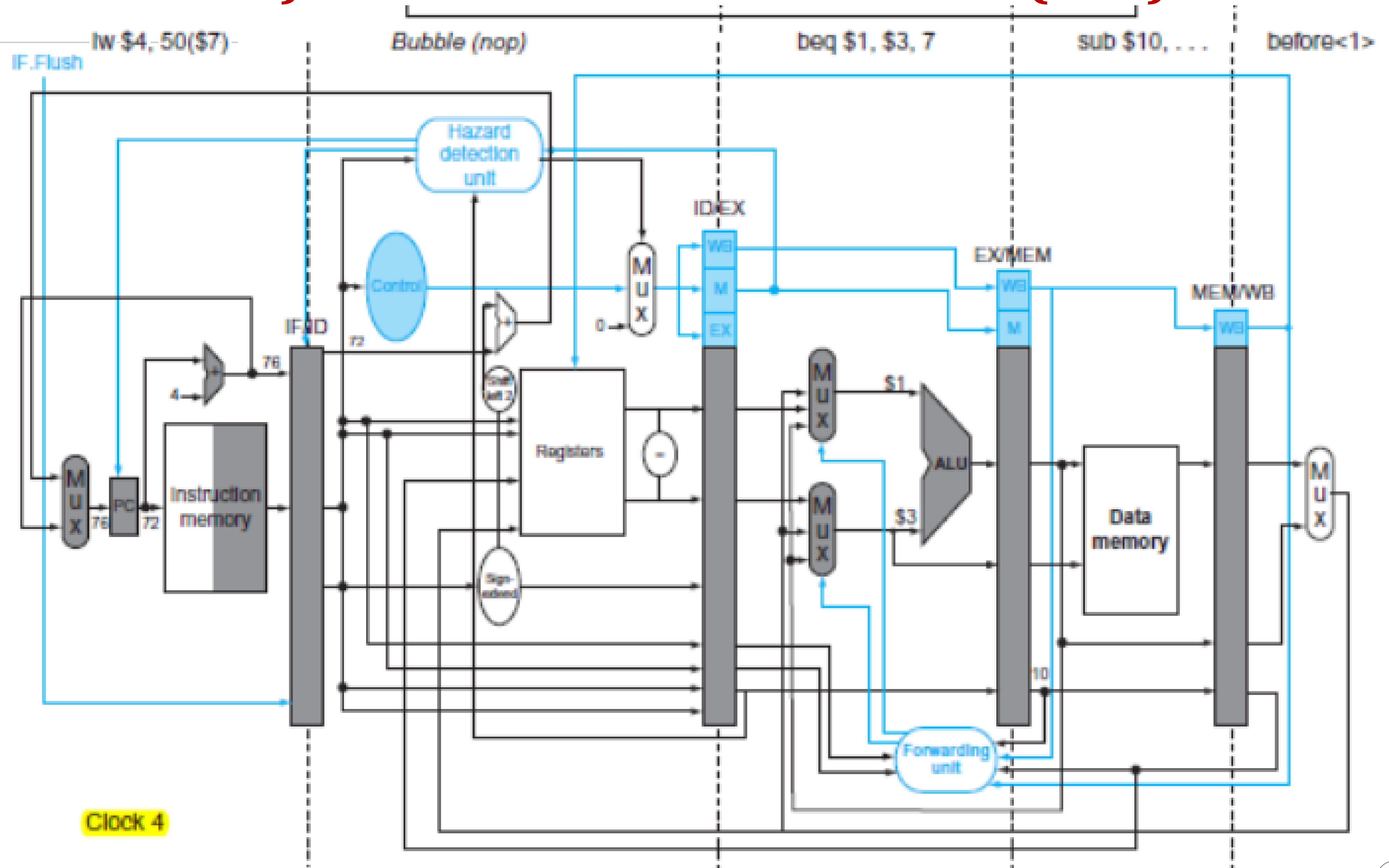
Control Hazards: 2) Predict – Branch Not Taken (Ctd.)

Example:



Control Hazards: 2) Predict – Branch Not Taken (Ctd.)

Example:



Control Hazards: 2) Predict - Dynamic

- **Dynamic Branch Prediction**

- In deeper pipelines, the branch penalty increases in terms of clock cycles and instructions lost.
- With more hardware, it is possible to predict branch behavior during program execution.

- **Prediction of branches at runtime using runtime information.**

- **One Implementation Approach:**

- **Branch Prediction Buffer or Branch History Table**

- A small memory that is indexed by the lower portion of the address of the branch instruction and that contains one or more bits indicating whether the branch was recently taken or not
 - So fetching begins in the predicted direction.

Control Hazards: 2) Predict – Dynamic Branch Prediction

Dynamic Branch Prediction

- The simple 1-bit prediction scheme has a **performance shortcoming**.
 - Even if branch is almost always taken, we can predict incorrectly twice, rather than once.
 - E.g. Loop branch that branches nine times in a row, then is not taken.
 - Misprediction happen on the first (can be corrected) and last loop iterations (can't avoid this).
 - Misprediction on the first iteration happens bcz the bit was flipped on prior execution of the last iteration of the loop.
 - Thus, the prediction accuracy for this branch that is taken 90% of the time is only 80%.
- **Ideally, the accuracy of the predictor should match the taken branch frequency.**

Control Hazards: 2) Predict – Dynamic Branch Prediction

Dynamic Branch Prediction

2-bit Prediction Schemes

- A prediction must be wrong twice before it is changed.
- Branch prediction buffer can be implemented as a special small buffer accessed with the instruction address during the IF pipeline Stage.
- If the prediction is taken then Fetching of new instruction begins from the target as soon as the PC is known (it can be as early as the ID stage)
 - Two Solutions:
 - **Delayed Branches** Approach
 - Use **Cache** to hold destination PC or Destination Instruction using a **Branch Target Buffer**
 - A structure that caches the destination PC or destination instruction for a branch. It is usually organized as a cache with tags, making it more costly than a simple prediction buffer.

Control Hazards: 2) Predict – Dynamic Branch Prediction

Dynamic Branch Prediction 2-bit Prediction Schemes

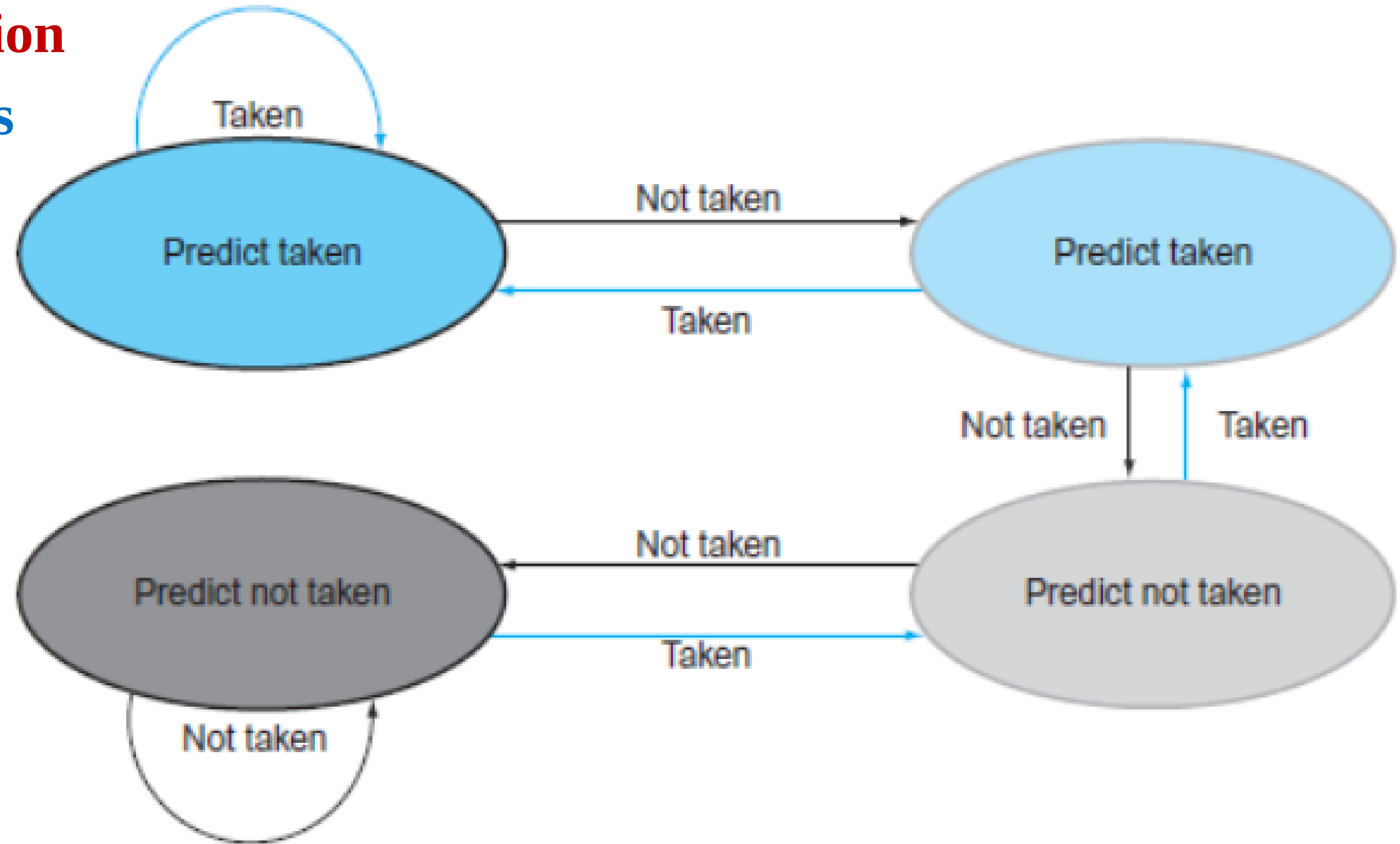


FIGURE 4.63 The states in a 2-bit prediction scheme. By using 2 bits rather than 1, a branch that strongly favors taken or not taken—as many branches do—will be mispredicted only once. The 2 bits are used

Control Hazards: 2) Predict – Dynamic Branch Prediction

Dynamic Branch Prediction

Other 2-bit Prediction Schemes

- Utilize both **local** branch, and the **global behavior** of recently executed branches.
- **Correlating Predictors**
 - A branch predictor that combines local behavior of a particular branch and global information about the behavior of some recent number of executed branches.
- **Tournament Predictors**
 - A branch predictor with multiple predictions for each branch and a selection mechanism that chooses which predictor to enable for a given branch.

Control Hazards: 3) Delayed Branches

Delayed Decision or Delayed Branches

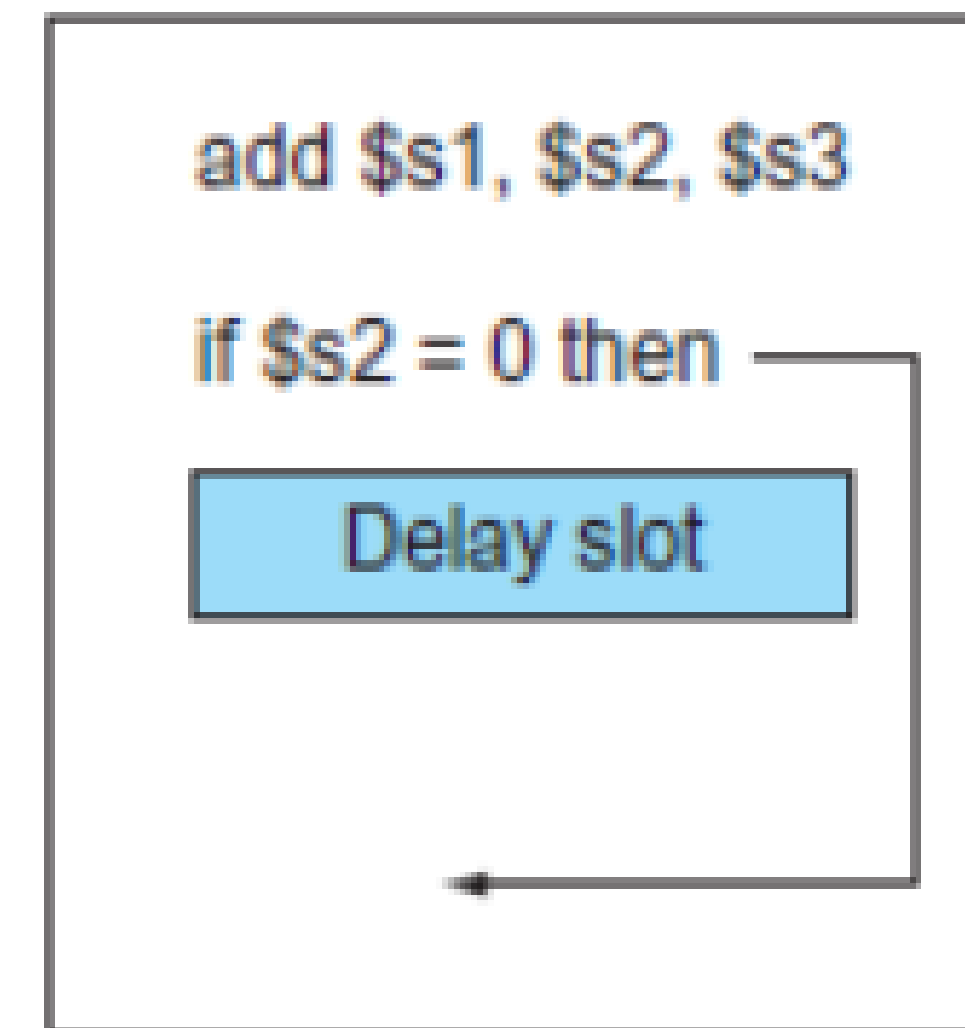
- Avoid control hazard by redefining the branch
- A delayed Branch always executes the following instruction, but the second instruction following the branch will be affected by the branch.
- **Branch Delay Slot:** The slot directly after a delayed branch instruction, which in the MIPS architecture is filled by an instruction that does not affect the branch.
- Compilers and Assemblers fill the Branch Delay Slot
- Limitations on Delayed Branch Scheduling:
 - The restrictions on the instructions that are scheduled into the delay slots
 - Limited prediction ability at compile time whether a branch is likely to be taken or not.

Control Hazards: 3) Delayed Branches

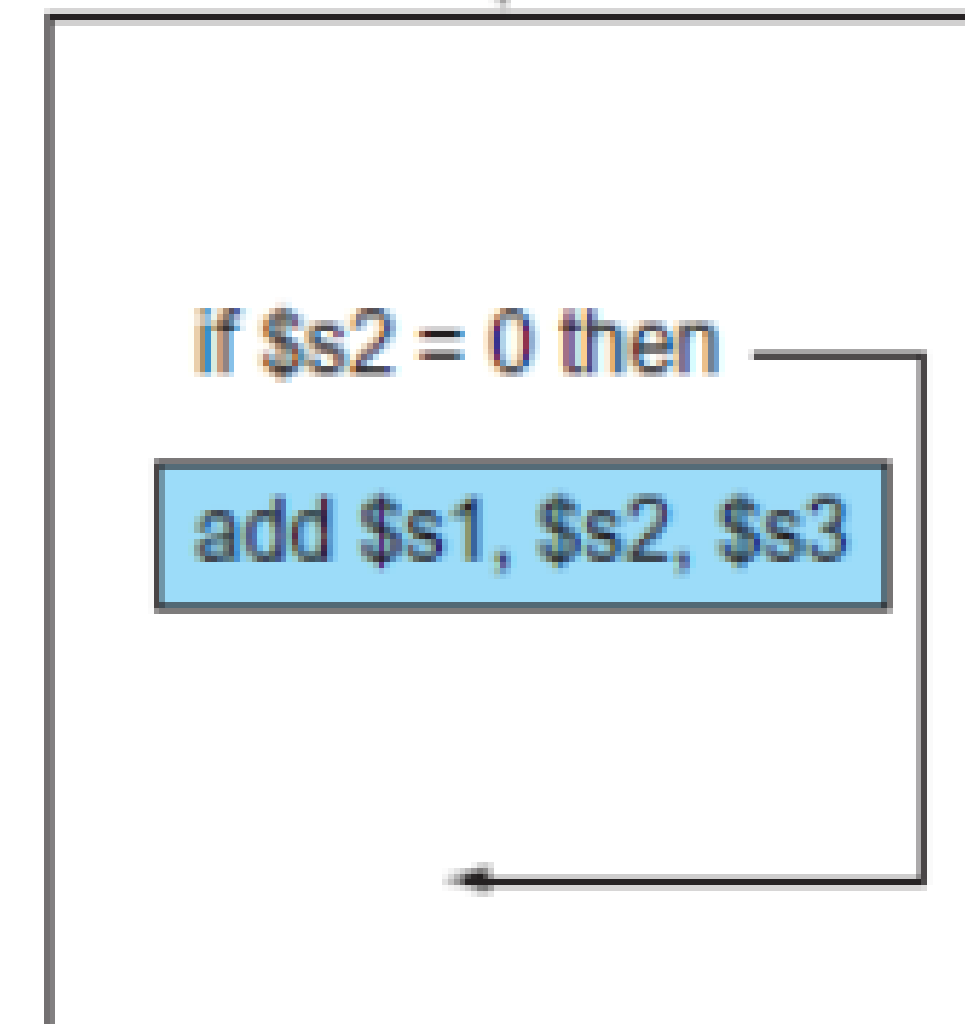
Delayed Decision or Delayed Branches

- Three ways in which the branch delay slot can be scheduled.
 - With an independent instruction (best option)

a. From before



Becomes

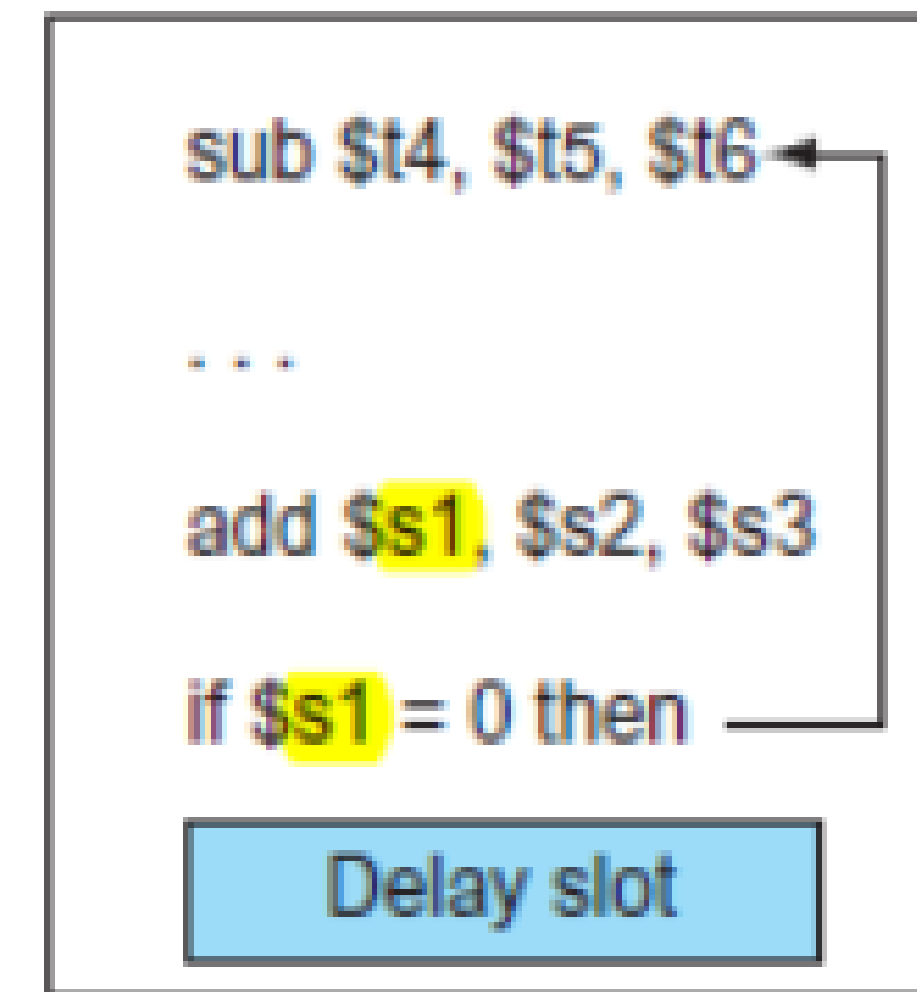


Control Hazards: 3) Delayed Branches

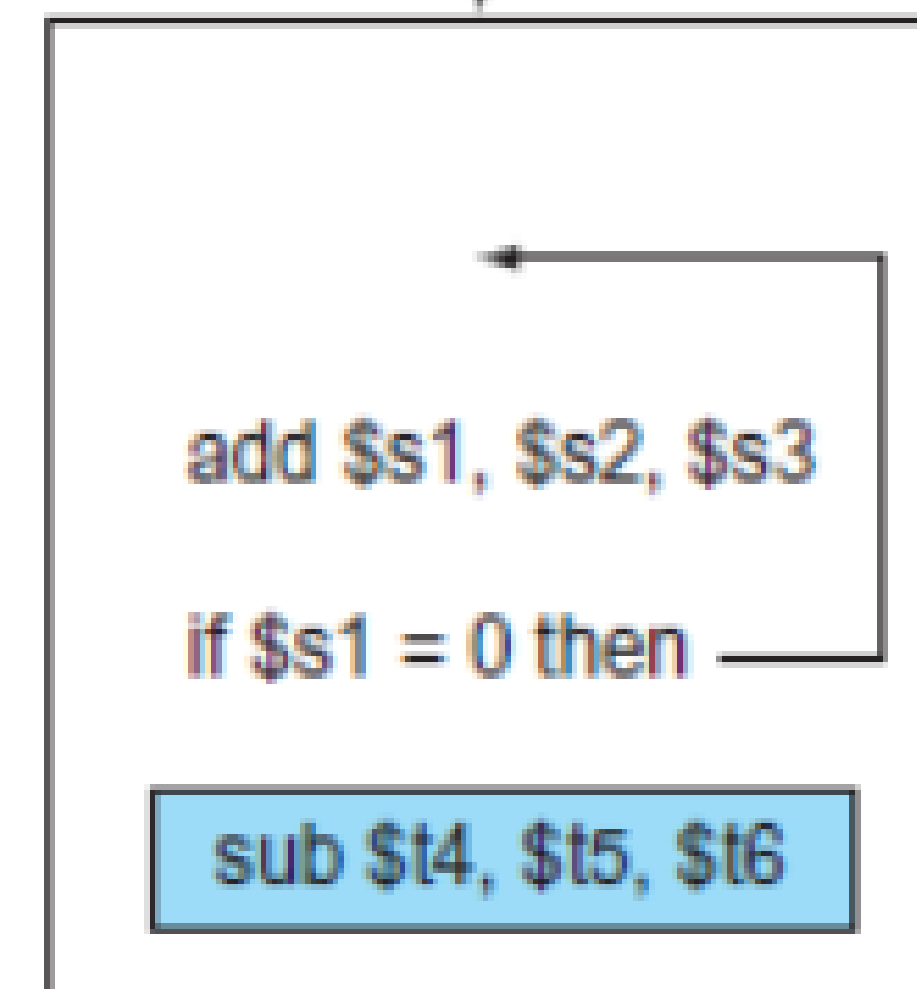
Delayed Decision or Delayed Branches

- Three ways in which the branch delay slot can be scheduled.
 - Data dependency to earlier instruction prevents its use after the branch.
 - Strategy (b) is preferred when the branch is taken with high probability e.g. loop branch
 - Assumption: It must be OK to execute sub instr. Even if the branch goes in the unexpected direction.

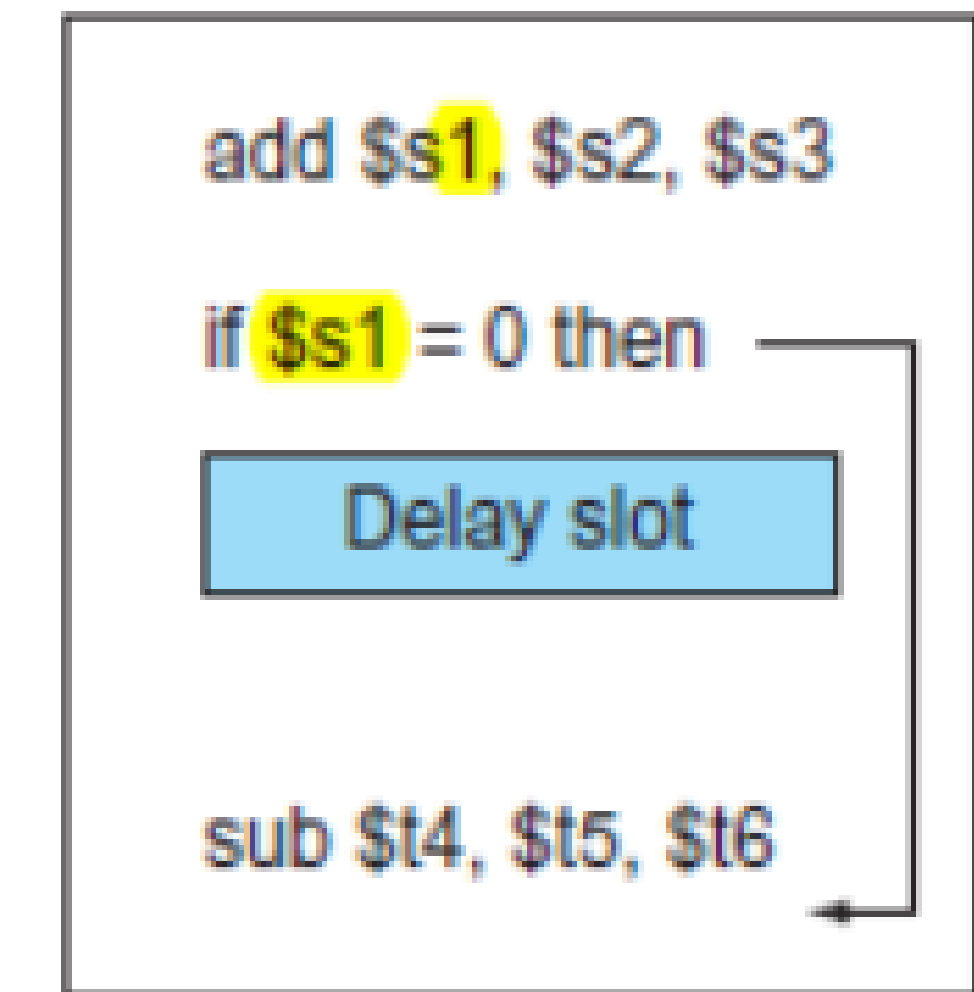
b. From target



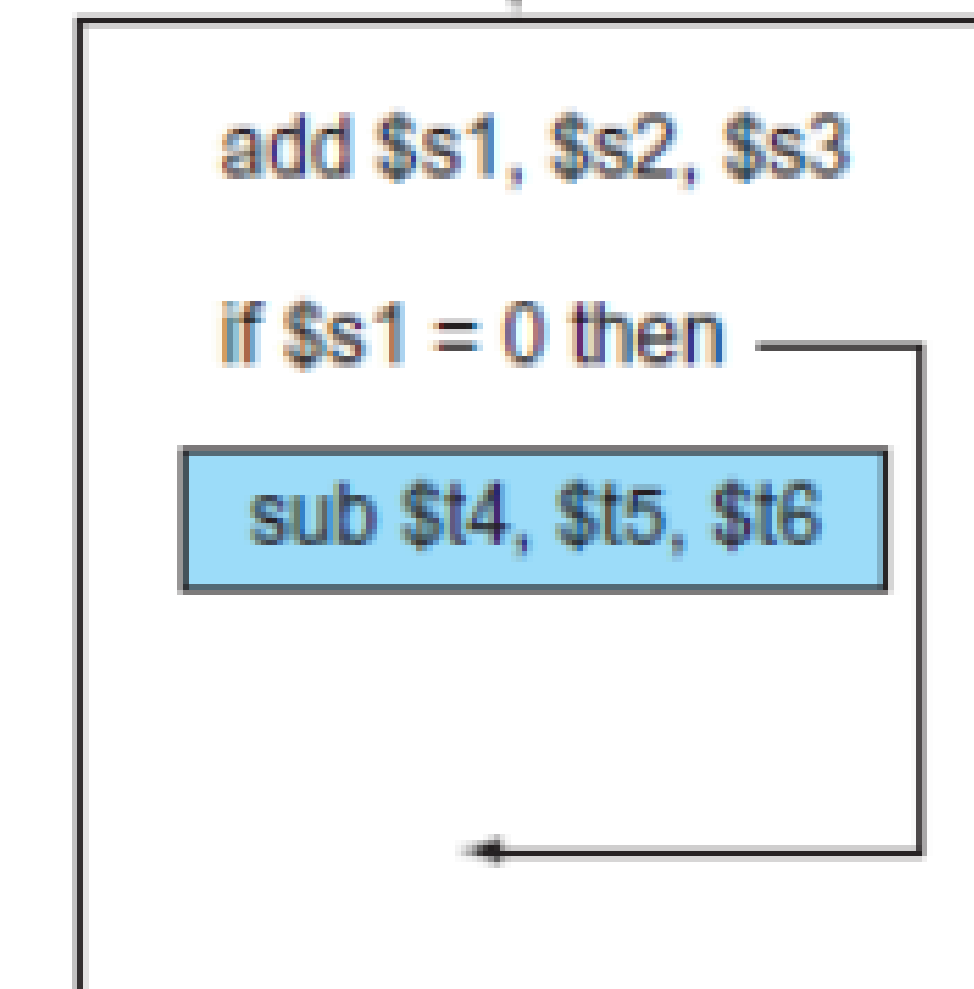
Becomes



c. From fall-through



Becomes



Control Hazards Summary

- **Stalling**

- Too costly to waste 1 clock cycle on every branch instruction in optimized datapath.

- **Predictions**

- Static Prediction: Simple technique used in simpler pipelines.
- Dynamic Prediction:
 - Expensive in terms of extra hardware required
 - Prediction Accuracy can go above 90%
 - Mainly used these days

- **Delayed Branches**

- Insert an independent instruction in the pipeline after the branch instead of stall.
- In MIPS, branches are always delayed.